

OD Estimation on the Sioux Falls Network

Methodology, Data Pipelines, and Machine-Learning Benchmarking

Implementation: `src/data/`, `src/ml/`, `generate_od_dataset.py`, `generate_turning_counts.py`, `tr`

July 14, 2026

Abstract

This document describes the full pipeline for origin–destination (OD) estimation on the Sioux Falls network, organized in three chapters: (i) synthetic OD matrix generation and evaluation, (ii) probabilistic turning-count observation via a route-choice-aware linear assignment operator, and (iii) machine-learning model development and benchmarking for the inverse problem. Notation is fixed throughout. The synthetic generator builds a distance-dependent gravity prior on full off-diagonal support, draws Dirichlet samples for compositional diversity, applies Gamma heavy-tailed reweighting for realistic flow skewness, scales demand, and balances with iterative proportional fitting (IPF). Candidate matrices are then accepted or rejected by dataset-level quality filters (sparsity, cell-wise entropy, pairwise Frobenius diversity, correlation bounds, and trip-length consistency). Turning counts are generated by K -shortest-path enumeration, multinomial logit route choice, and a fractional assignment matrix $\mathbf{A}_{\text{turn}}$, with optional Poisson observation noise. Implementations live under `src/data/` and `src/ml/`. The design supports datasets of up to 10^6 samples using batched, memory-mapped I/O.

Contents

1	Synthetic Data Generation and Evaluation	3
1.1	Problem setting and pipeline overview	3
1.1.1	The OD matrix	3
1.1.2	Why synthetic data?	3
1.1.3	End-to-end pipeline	4
1.1.4	Notation summary	4
1.1.5	Codebase organization	4
1.2	Structural constraints and design rationale	5
1.2.1	Support and hard constraints	5
1.3	Generative methodology	6
1.3.1	Stage 1: Base OD analysis	6
1.3.2	Stage 2: Distance-dependent gravity prior	6
1.3.3	Stage 3: Dirichlet sampling on the simplex	7
1.3.4	Stage 4: Gamma heavy-tailed reweighting	7
1.3.5	Stage 5: Demand scaling and optional marginal perturbation	8
1.3.6	Stage 6: Iterative proportional fitting (IPF)	8
1.4	Quality filters (dataset acceptance)	9
1.5	Evaluation metrics and hyperparameter selection	10
1.5.1	Per-matrix metrics	10
1.5.2	Dataset-level metrics and hyperparameter search	10
1.6	Scalability and implementation reference	11
2	Turning-Count Observation and Dataset Construction	12
2.1	Network representation and turning movements	12
2.1.1	Binary vs. probabilistic route assignment	12
2.2	Route enumeration and probabilistic assignment	13
2.2.1	OD pair indexing	13
2.2.2	Route catalog: K shortest simple paths	13
2.2.3	Multinomial logit route choice	13
2.3	Fractional assignment matrix and forward model	13
2.3.1	Inputs, outputs, and indexing conventions	13
2.3.2	Turn incidence from a route path	14
2.3.3	Column assembly for each OD pair	14
2.3.4	Binary baseline for comparison	14
2.3.5	Validation checks	15
2.3.6	Forward model	15
2.3.7	Identifiability and rank analysis	15
2.4	Observation noise, dataset assembly, and scalability	15
2.4.1	Observation noise models	15
2.4.2	End-to-end pipeline and batched generation	16
2.4.3	Validation, sensitivity, and evaluation highlights	16

2.5	Implementation reference	16
3	OD Estimation: Information Analysis, Models, and Benchmarking	18
3.1	Inverse problem and design principle	18
3.2	Dataset and information content analysis	18
3.3	Model zoo and selection rationale	19
3.4	Physics constraints, loss functions, and evaluation metrics	19
3.4.1	Loss functions	19
3.5	Training protocol, robustness, and explainability	20
3.6	Benchmark leaderboard and deliverables	20
3.7	Future work and limitations	21
	Conclusion	22

Chapter 1

Synthetic Data Generation and Evaluation

1.1 Problem setting and pipeline overview

1.1.1 The OD matrix

Let $N = 24$ be the number of traffic analysis zones in Sioux Falls. The *base* OD matrix

$$\mathbf{T}^{\text{base}} = (T_{ij}^{\text{base}})_{i,j=1}^N \in \mathbb{R}_{\geq 0}^{N \times N} \quad (1.1)$$

records daily trips from origin zone i to destination zone j . We observe a single matrix `EstimatedODMatrix.npy`; the goal is to train models that recover OD demand from turning-count observations.

Marginals and grand total.

$$p_i^{\text{base}} = \sum_{j=1}^N T_{ij}^{\text{base}} \quad (\text{productions}), \quad (1.2)$$

$$a_j^{\text{base}} = \sum_{i=1}^N T_{ij}^{\text{base}} \quad (\text{attractions}), \quad (1.3)$$

$$G = \sum_{i,j} T_{ij}^{\text{base}} = \sum_i p_i^{\text{base}} = \sum_j a_j^{\text{base}}. \quad (1.4)$$

1.1.2 Why synthetic data?

Machine learning requires many labeled examples. We generate K_{samples} synthetic matrices $\{\mathbf{T}^{\text{syn},(k)}\}_{k=1}^{K_{\text{samples}}}$ that:

- preserve total demand G and forbid intrazonal trips ($T_{ii} = 0$);
- admit full off-diagonal support $i \neq j$, guided by a gravity-style prior;
- exhibit heavy-tailed cell flows (many tiny OD pairs, few large ones);
- pass dataset-level diversity and realism filters before inclusion;
- pair with turning counts via a known forward model for supervised training.

Chapter 1: $\mathbf{T}^{\text{base}} \rightarrow \text{prior } (T + \varepsilon) \rightarrow \text{Dirichlet} \rightarrow \text{Gamma weighting} \rightarrow \text{scale} \rightarrow \text{IPF} \rightarrow \text{quality filters} \rightarrow \{\mathbf{T}^{\text{syn},(k)}\}$
Chapter 2: $\mathbf{T}^{\text{syn},(k)} \rightarrow \mathbf{y}^{(k)} \rightarrow \mathbf{x}^{(k)} = \mathbf{A}_{\text{turn}}\mathbf{y}^{(k)} + \boldsymbol{\epsilon}^{(k)}$ (K -paths + logit + fractional $\mathbf{A}_{\text{turn}}$)
Chapter 3: $\mathbf{X}^{\text{obs}} \rightarrow f(\cdot) \rightarrow \hat{\mathbf{Y}}$ (information analysis $\rightarrow$ baselines $\rightarrow$ ML benchmark)

Figure 1.1: Three-chapter pipeline from base OD to ML-ready $(\mathbf{X}, \mathbf{Y})$ pairs.

Table 1.1: Primary notation used throughout this document.

Symbol	Meaning
$\mathbf{T}^{\text{base}}$	Observed base OD matrix
$\mathbf{T}^{\text{syn}}, \mathbf{T}^{\text{syn},(k)}$	Synthetic OD (single / k -th replicate)
$\mathbf{p}^{\text{base}}, \mathbf{a}^{\text{base}}$	Base productions and attractions
$\mathbf{p}^{\text{tgt}}, \mathbf{a}^{\text{tgt}}$	IPF target marginals (sum to G)
$\mathbf{p}_i^{\text{syn}}, \mathbf{a}_j^{\text{syn}}$	Achieved marginals of $\mathbf{T}^{\text{syn}}$
$\mathbf{p}^{\text{prior}}, \varepsilon_{ij}$	Gravity prior probabilities / distance-dependent additive mass
d_{ij}, β, λ	Inter-zone distance; prior scale and decay length
$\tilde{\mathbf{p}}^{\text{Dir}}, \mathbf{w}$	Dirichlet sample; Gamma weights
$\mathbf{p}^{\text{prob}}$	Sampling prior on support $ \mathcal{S} = N(N - 1) = 552$
$\mathbf{S}, \mathbf{X}^{(t)}$	IPF seed matrix / iterate at step t
$\mathbf{M}$	Off-diagonal support mask ($M_{ij} = \mathbb{1}[i \neq j]$)
$\mathbf{y}, \mathbf{Y}$	Flattened OD vector / batch ($N^2 = 576$)
$\mathbf{x}, \mathbf{X}$	Turning-count vector / batch ($M = 178$); clean or noisy
$\mathbf{x}^{\text{clean}}, \mathbf{x}^{\text{obs}}$	Clean turning counts / noisy observations
$\mathbf{A}_{\text{turn}}$	Fractional turning assignment matrix (178×576)
$\mathbf{A}_{\text{turn}}^{\text{bin}}$	Binary (shortest-path) assignment matrix
$\mathcal{R}_{ij}$	Set of enumerated routes for OD pair (i, j)
K	Number of shortest simple paths per OD pair
θ	Logit sensitivity (cost scale in route choice)
c_r, U_r, P_r	Route cost, utility, choice probability
$\mathbf{b}_r$	Binary turn-incidence vector for route r
α, δ	Dirichlet concentration / marginal perturbation level
a_γ, b_γ	Gamma shape and scale for heavy-tailed weighting
H_{ij}	Cell-wise Shannon entropy across the dataset

1.1.3 End-to-end pipeline

1.1.4 Notation summary

1.1.5 Codebase organization

Separating **data generation** from **model estimation** is advisable: the two stages have different dependencies (NetworkX vs. PyTorch/XGBoost), different scaling bottlenecks, and different release cadences.

Why separate packages.

- **Dependency isolation:** data generation requires no GPU/PyTorch; ML stage optionally does.
- **Memory profile:** Chapter 2 uses memory-mapped batch I/O; Chapter 3 uses train/val/test arrays.
- **Testing:** data-pipeline tests and ML tests run independently.
- **Clarity:** inverse-problem code does not import route enumeration modules.

Table 1.2: Recommended `src/` package layout.

Package	Responsibility	Entry scripts
<code>src/data/</code>	Synthetic OD, turning counts, $\mathbf{A}_{\text{turn}}$	<code>generate_od_dataset.py</code> , <code>generate_turning_counts.py</code>
<code>src/ml/</code>	Information analysis, training, benchmarking	<code>train_od_estimator.py</code>
<code>src/__init__.py</code>	Shared constants ($N = 24$, paths)	—

1.2 Structural constraints and design rationale

This chapter treats OD synthesis as a constrained random measure on zone pairs. Four modelling decisions shape the generator: (i) which cells may carry flow, (ii) how a prior encodes spatial structure, (iii) how samples explore the probability simplex, and (iv) how dataset-level filters enforce diversity and realism.

1.2.1 Support and hard constraints

Unlike an earlier design that restricted sampling to cells with positive base flow, the current support is *full off-diagonal*:

$$\mathcal{S} = \{(i, j) : i \neq j\}, \quad |\mathcal{S}| = N(N - 1) = 552, \quad M_{ij} = \mathbb{1}[i \neq j]. \quad (1.5)$$

Hard constraints enforced for every synthetic replicate are

$$T_{ii}^{\text{syn}} = 0 \quad (\text{no intrazonal trips}), \quad (1.6)$$

$$\sum_{i,j} T_{ij}^{\text{syn}} = G \quad (\text{conserved demand}), \quad (1.7)$$

$$T_{ij}^{\text{syn}} \geq 0 \quad (\text{nonnegative flows}). \quad (1.8)$$

Allowing $i \neq j$ cells that were zero in $\mathbf{T}^{\text{base}}$ is deliberate: ML estimators of underdetermined OD problems must not be trained under an artificially narrow support that coincides with one observed matrix. The gravity prior of Section 1.3.2 then softly concentrates mass on short, historically active pairs without forbidding exploration of longer links.

Table 1.3: Generative approaches compared (Sioux Falls). The production pipeline is prior + Dirichlet + Gamma + IPF + quality filters.

Criterion	Legacy noise	Dirichlet+IPF only	Current (full)
Support	$\{T^{\text{base}} > 0\}$	$\{T^{\text{base}} > 0\}$	Full $i \neq j$ + distance prior
Composition	$\eta_{ij} \sim \mathcal{U}(0.8, 1.2)$	$\text{Dir}(\alpha \mathbf{p})$	Dirichlet then Gamma reweight
Skewness	Mild	Mild–moderate	Heavy-tailed (many tiny / few large)
Diversity control	Weak	α only	$\alpha, (a_\gamma, b_\gamma)$, filters
Typical ρ vs. base	≈ 0.97	≈ 0.74 ($\alpha = 500$)	Bounded by filter band $[\rho_{\min}, \rho_{\max}]$

Table 1.4: Pros and cons of support choices.

Choice	Pros	Cons
Support $\{T^{\text{base}} > 0\}$	Matches observed sparsity; stable IPF	Cannot invent missing OD pairs; ML may overfit support
Full off-diagonal $\mathcal{S}$	Richer label space; consistent with zoning	Needs a prior; more near-zero cells

1.3 Generative methodology

The production generator proceeds in six stages:

$$\begin{aligned} \mathbf{T}^{\text{base}} &\xrightarrow{\text{prior}} \mathbf{p}^{\text{prior}} \xrightarrow{\text{Dir}} \tilde{\mathbf{p}}^{\text{Dir}} \xrightarrow{\text{Gamma}} \mathbf{p}^{\text{prob}} \\ &\xrightarrow{\text{scale}} \mathbf{S} \xrightarrow{\text{IPF}} \mathbf{T}^{\text{syn,cand}} \xrightarrow{\text{filters}} \{\mathbf{T}^{\text{syn},(k)}\}. \end{aligned} \quad (1.9)$$

1.3.1 Stage 1: Base OD analysis

Given $\mathbf{T}^{\text{base}}$, compute grand total $G = \sum_{i,j} T_{ij}^{\text{base}}$, productions $\mathbf{p}^{\text{base}}$ and attractions $\mathbf{a}^{\text{base}}$, and descriptive statistics (sparsity, top OD pairs). Total demand G is held fixed in subsequent sampling; only the *composition* of trips is randomized. Implementation: `src/data/od_analysis.py`.

1.3.2 Stage 2: Distance-dependent gravity prior

Let d_{ij} be Euclidean distance between zone centroids (junction coordinates from the SUMO network). Define additive prior mass

$$\varepsilon_{ij} = \begin{cases} \beta e^{-d_{ij}/\lambda}, & i \neq j, \\ 0, & i = j, \end{cases} \quad (1.10)$$

and the prior trip table

$$T_{ij}^{\text{prior}} = T_{ij}^{\text{base}} + \varepsilon_{ij}, \quad p_{ij}^{\text{prior}} = \frac{T_{ij}^{\text{prior}}}{\sum_{k,\ell} T_{k\ell}^{\text{prior}}}. \quad (1.11)$$

Defaults: $\beta = 1.0$, $\lambda = 500$ m (order of the median inter-zone distance on Sioux Falls). The exponential decay encodes a gravity-style preference for shorter trips while retaining the observed base structure in T^{base} . Importantly, G used for scaling remains $\sum T^{\text{base}}$, not $\sum T^{\text{prior}}$ —the prior only shapes probabilities.

Why this prior. Base-only normalization T^{base}/G leaves historically inactive OD pairs at probability zero, so Dirichlet never places mass there. Adding $\varepsilon_{ij} > 0$ opens full off-diagonal support in a continuous, distance-aware manner without imposing a hard cut-off.

Table 1.5: Pros and cons of prior constructions.

Method	Pros	Cons
T^{base}/G only	Faithful to survey; simple	Zeros remain forever; no gravity signal
Uniform off-diagonal	Maximal exploration	Ignores geography and base hierarchy
$T + \varepsilon$ with (1.10)	Smooth full support; short trips preferred	Requires (β, λ) ; mild leakage onto long OD pairs

1.3.3 Stage 3: Dirichlet sampling on the simplex

Let $\mathbf{p}_S^{\text{prior}} \in \Delta^{|S|-1}$ be the prior restricted to support cells. Draw

$$\tilde{\mathbf{p}}^{\text{Dir}} \sim \text{Dirichlet}(\alpha \mathbf{p}_S^{\text{prior}}). \quad (1.12)$$

The Dirichlet density on the probability simplex Δ^{d-1} with parameter $\alpha = (\alpha_1, \dots, \alpha_d)$, $\alpha_k > 0$, is

$$f(\mathbf{p}) = \frac{\Gamma(\sum_k \alpha_k)}{\prod_k \Gamma(\alpha_k)} \prod_{k=1}^d p_k^{\alpha_k-1}, \quad p_k > 0, \sum_k p_k = 1. \quad (1.13)$$

With the proportional parameterization $\alpha_k = \alpha p_k^{\text{prior}}$, the mean is

$$\mathbb{E}[\tilde{p}_k^{\text{Dir}}] = p_k^{\text{prior}}, \quad (1.14)$$

and the variance of each coordinate decreases as α increases:

$$\text{Var}(\tilde{p}_k^{\text{Dir}}) = \frac{p_k^{\text{prior}}(1 - p_k^{\text{prior}})}{\alpha + 1}. \quad (1.15)$$

Thus α is a single scalar knob trading off realism against compositional diversity.

Why Dirichlet. Trip tables are compositional: shares must sum to one under fixed G . The Dirichlet is the conjugate / natural distribution on that simplex; proportional means preserve the prior ranking of OD pairs in expectation while allowing controlled random reallocation of mass. Independent Gaussian noise on cells would violate the simplex (or require clumsy post-normalization) and ignore the compositional constraint.

Table 1.6: Dirichlet concentration α : empirical trade-offs on Sioux Falls ($K_{\text{samples}} = 5000$, legacy support study; production uses the same qualitative regime).

α	Mean ρ vs. base	Pairwise diversity	Recommendation
100	lower (~ 0.65)	highest	exploratory / high-diversity ML
250	moderate	high	alternative if $\alpha = 500$ too conservative
500	≈ 0.74	≈ 545	default concentration
1000	higher	moderate	closer to base
2000	highest ($\sim 0.90+$)	lowest	ablation only

Table 1.7: Pros and cons of compositional sampling methods.

Method	Pros	Cons
Dirichlet (1.12)	Correct support (simplex); mean = prior; one knob α	Does not alone create extreme flow skewness
Independent lognormal cells + normalize	Easy heavy tails	Normalization couples cells awkwardly; hard to set means
Additive on $\mathbf{T}^{\text{base}}$	Gaussian Simple	Can leave simplex/negativity; clones of base at high SNR

1.3.4 Stage 4: Gamma heavy-tailed reweighting

Empirical OD matrices are highly skewed: most zone pairs carry small flows, while a few corridors dominate. Dirichlet draws alone tend to be smoother than this. After Dirichlet, we therefore apply independent positive weights

$$w_{ij} \sim \text{Gamma}(a_\gamma, b_\gamma), \quad i \neq j, \quad (1.16)$$

in the shape–scale parameterization with density

$$f(w) = \frac{1}{\Gamma(a_\gamma) b_\gamma^{a_\gamma}} w^{a_\gamma-1} e^{-w/b_\gamma}, \quad w > 0, \quad (1.17)$$

mean $\mathbb{E}[w] = a_\gamma b_\gamma$ and variance $\text{Var}(w) = a_\gamma b_\gamma^2$. Reweight and renormalize:

$$p_{ij}^{\text{prob}} = \frac{w_{ij} \tilde{p}_{ij}^{\text{Dir}}}{\sum_{k \neq \ell} w_{k\ell} \tilde{p}_{k\ell}^{\text{Dir}}}, \quad p_{ii}^{\text{prob}} = 0. \quad (1.18)$$

Defaults: $a_\gamma = 0.5$, $b_\gamma = 1.0$. For $a_\gamma < 1$, the Gamma density diverges as $w \rightarrow 0^+$ and retains a long right tail, yielding many tiny OD shares and occasional large weights—exactly the heavy-tailed pattern desired for synthetic tables. Disable with `--no-gamma-mask`.

Why Gamma (not LogNormal / Pareto alone). Gamma provides strictly positive weights with a single, interpretable shape controlling near-zero mass. LogNormal is also heavy-tailed but less flexible near zero for the same two-parameter budget; Pareto concentrates too aggressively on extremes and can destabilize IPF. Mixing Dirichlet (composition) with Gamma (magnitude skew) separates “where mass can move” from “how uneven magnitudes are.”

Table 1.8: Pros and cons of heavy-tailed reweighting.

Method	Pros	Cons
None (Dirichlet only)	Stable; fewer hyperparameters	Over-smooth OD hierarchy
Gamma $a_\gamma < 1$	Many tiny / few large flows; positive weights	Extra (a_γ, b_γ) ; can dilute base pattern
LogNormal	Classic heavy tail	Weaker near-zero density control
Pareto / power-law	Extreme corridors	Can break IPF / correlation filters

1.3.5 Stage 5: Demand scaling and optional marginal perturbation

Form the IPF seed

$$S_{ij} = G p_{ij}^{\text{prob}}, \quad S_{ii} = 0. \quad (1.19)$$

Optional multiplicative shocks on zone totals ($\delta \geq 0$; production often uses $\delta = 0$ when filters already control diversity):

$$\tilde{p}_i = p_i^{\text{base}} (1 + \xi_i^p), \quad \xi_i^p \sim \mathcal{U}(-\delta, \delta), \quad (1.20)$$

$$\tilde{a}_j = a_j^{\text{base}} (1 + \xi_j^a), \quad \xi_j^a \sim \mathcal{U}(-\delta, \delta), \quad (1.21)$$

$$p_i^{\text{tgt}} = G \tilde{p}_i / \sum_{i'} \tilde{p}_{i'}, \quad a_j^{\text{tgt}} = G \tilde{a}_j / \sum_{j'} \tilde{a}_{j'}. \quad (1.22)$$

With $\delta = 0$, targets equal the base marginals (rescaled to G).

1.3.6 Stage 6: Iterative proportional fitting (IPF)

IPF (Furness / RAS) balances the seed to the target marginals. With $\mathbf{X}^{(0)} = \mathbf{S}$,

$$X_{ij}^{(t+\frac{1}{2})} = X_{ij}^{(t)} \cdot \frac{p_i^{\text{tgt}}}{\sum_{j'} X_{ij'}^{(t)}}, \quad (1.23)$$

$$X_{ij}^{(t+1)} = X_{ij}^{(t+\frac{1}{2})} \cdot \frac{a_j^{\text{tgt}}}{\sum_{i'} X_{i'j}^{(t+\frac{1}{2})}}. \quad (1.24)$$

Table 1.9: Pros and cons of marginal perturbation.

Setting	Pros	Cons
$\delta = 0$	Stable productions/attractions; clear ablation	Less zone-level diversity
$\delta > 0$ (e.g. 0.20)	Realistic zone growth/decline	Can fight gravity prior / trip-length filters

Stop when marginal error $e^{(t)} < \tau = 10^{-3}$ or after $t_{\max} = 1000$ iterations; then $\mathbf{T}^{\text{syn,cand}} = \mathbf{X}^{(t)} \odot \mathbf{M}$ with zero diagonal.

Table 1.10: Pros and cons of balancing methods.

Method	Pros	Cons
IPF / Furness	Classic; fast; preserves zeros of the seed pattern up to scaling	Requires feasible marginals; iterative
Entropy maximization	Elegant dual interpretation	Heavier numerically for 10^6 samples
No balancing	Cheap	Inconsistent zone totals

Algorithm 1: Prior + Dirichlet + Gamma + IPF (one candidate)

1. Build $T^{\text{prior}} = T^{\text{base}} + \varepsilon$ with (1.10)–(1.11); mask $\mathbf{M}$
2. Draw $\tilde{\mathbf{p}}^{\text{Dir}} \sim \text{Dir}(\alpha \mathbf{p}_S^{\text{prior}})$
3. Sample $w_{ij} \sim \text{Gamma}(a_\gamma, b_\gamma)$; form $\mathbf{p}^{\text{prob}}$ by (1.18)
4. Seed $\mathbf{S} = G \text{reshape}(\mathbf{p}^{\text{prob}})$; build targets $(\mathbf{p}^{\text{tgt}}, \mathbf{a}^{\text{tgt}})$
5. $\mathbf{T}^{\text{syn,cand}} \leftarrow \text{IPF}(\mathbf{S}, \mathbf{p}^{\text{tgt}}, \mathbf{a}^{\text{tgt}})$; project onto $\mathbf{M}$

Figure 1.2: Candidate generator (`src/data/od_generator.py`, `dirichlet_sampler.py`, `sparse_mask.py`).

1.4 Quality filters (dataset acceptance)

Candidate generation alone does not guarantee a useful *dataset*: replicates can cluster, clone $\mathbf{T}^{\text{base}}$, or violate trip-length realism. We therefore generate an oversized pool of size

$$K_{\text{pool}} = \lceil \kappa K_{\text{samples}} \rceil \quad (\text{default } \kappa = 10) \quad (1.25)$$

and accept a subset of size K_{samples} by sequential filters (implementation: `src/data/quality_filters.py`).

Sparsity band. Let $s(\mathbf{T}) = \frac{1}{N^2} \sum_{i,j} \mathbb{1}[T_{ij} \leq 10^{-12}]$. Require $s_{\min} \leq s(\mathbf{T}^{\text{syn,cand}}) \leq s_{\max}$ (defaults 0 and 0.95).

Correlation band. With $\rho = \text{corr}(\text{vec}(\mathbf{T}^{\text{base}}), \text{vec}(\mathbf{T}^{\text{syn,cand}}))$, require $\rho_{\min} \leq \rho \leq \rho_{\max}$ (defaults 0.30 and 0.95) to reject both clones and wild outliers.

Trip-length consistency. Flow-weighted mean trip length

$$\bar{\ell}(\mathbf{T}) = \frac{\sum_{i,j} T_{ij} d_{ij}}{\sum_{i,j} T_{ij}} \quad (1.26)$$

must satisfy $|\bar{\ell}(\mathbf{T}^{\text{syn,cand}}) - \bar{\ell}(\mathbf{T}^{\text{base}})| / \bar{\ell}(\mathbf{T}^{\text{base}}) \leq \eta_\ell$ (default $\eta_\ell = 0.35$).

Cell-wise entropy ranking (Constraint 3). For cell (i, j) across a batch $\{\mathbf{T}^{\text{syn},(k)}\}_{k=1}^K$, discretize flows into B bins and compute Shannon entropy

$$H_{ij} = -\sum_{b=1}^B \hat{\pi}_{ij,b} \log \hat{\pi}_{ij,b}, \quad \bar{H} = \frac{1}{|\mathcal{S}|} \sum_{(i,j) \in \mathcal{S}} H_{ij}. \quad (1.27)$$

Candidates are ranked to prefer increments in $\bar{H}$ (diverse marginal histograms per OD cell).

Pairwise Frobenius diversity (Constraint 4). Reject $\mathbf{T}$ if there exists an already accepted $\mathbf{T}'$ with

$$\|\mathbf{T} - \mathbf{T}'\|_F < \tau_F, \quad \|\mathbf{A}\|_F = \sqrt{\sum_{i,j} A_{ij}^2}. \quad (1.28)$$

If τ_F is unset, set $\tau_F = 0.35 \times \text{median}\{\|\mathbf{T}^{(a)} - \mathbf{T}^{(b)}\|_F\}$ over a sample of candidate pairs.

Table 1.11: Pros and cons of quality filters.

Filter	Pros	Cons
Sparsity / corr. bands	Cheap realism gates	Thresholds are network-specific
Trip-length check	Geography-aware acceptance	Needs distances; couples to prior
Cell entropy H_{ij}	Encourages rich per-cell variation	Binning choices; soft ranking
Frobenius diversity	Explicit anti-cloning of accept set	$O(K_{\text{acc}})$ checks; needs oversized pool
No filters	Maximum throughput	Redundant / unrealistic batches

1.5 Evaluation metrics and hyperparameter selection

1.5.1 Per-matrix metrics

For each $\mathbf{T}^{\text{syn}}$, with $d_{ij} = T_{ij}^{\text{syn}} - T_{ij}^{\text{base}}$:

$$\text{MAE} = \frac{1}{N^2} \sum_{i,j} |d_{ij}|, \quad \text{RMSE} = \sqrt{\frac{1}{N^2} \sum_{i,j} d_{ij}^2}, \quad (1.29)$$

$$\text{RelErr} = \frac{\sum_{i,j} |d_{ij}|}{G}, \quad \rho = \text{corr}(\text{vec}(\mathbf{T}^{\text{base}}), \text{vec}(\mathbf{T}^{\text{syn}})). \quad (1.30)$$

Marginal drift: $E_p = \frac{1}{N} \sum_i |(p_i^{\text{syn}} - p_i^{\text{base}})/(p_i^{\text{base}} + 10^{-9})|$ (and analogously E_a). Structural validity requires zero diagonal mass and nonnegative flows on $\mathcal{S}$.

1.5.2 Dataset-level metrics and hyperparameter search

For batch $\{\mathbf{T}^{\text{syn},(k)}\}$: pairwise diversity (L1 / Frobenius), per-OD mean/std/CV, demand consistency $\text{CV}_G \approx 0$, mean cell entropy $\bar{H}$, PCA explained variance, and KMeans inertia. For $K_{\text{samples}} > 10^5$, metrics use a random subset (default 10,000).

`src/data/hyperparameter_search.py` evaluates (α, δ) pairs and ranks by

$$\text{score} = \frac{\bar{d}}{200} + \overline{\text{CV}}_{\text{cell}} + \frac{n_{90\%}}{20} - 2 \max(0, \bar{\rho} - 0.85). \quad (1.31)$$

Recommended production defaults: $\alpha = 500$, $\delta = 0$ (when quality filters are on), $\beta = 1$, $\lambda = 500$, $a_\gamma = 0.5$, $b_\gamma = 1$, $\kappa = 10$.

1.6 Scalability and implementation reference

For $K_{\text{samples}} = 10^6$: multiprocessing, float32 storage (~ 2.3 GB), per-index RNG, in-place IPF, oversized candidate pool then filters, sampled evaluation. Candidate generation remains the throughput bottleneck; filters add a secondary pass over K_{pool} .

Table 1.12: Chapter 1: key modules and outputs (`src/data/`).

Module / script	Role
<code>generate_od_dataset.py</code>	CLI orchestrator (prior, Gamma, filters)
<code>od_analysis.py</code>	Base OD statistics
<code>od_probability.py</code>	Distance prior ε and $\mathbf{p}^{\text{prior}}$
<code>dirichlet_sampler.py</code>	Dirichlet draw + demand reconstruction
<code>sparse_mask.py</code>	Gamma heavy-tailed reweighting
<code>marginal_perturbation.py</code>	Target marginals
<code>ipf.py</code>	IPF balancing
<code>od_generator.py</code>	Parallel candidate generation
<code>quality_filters.py</code>	Accept / reject pool filters
<code>od_metrics.py</code>	Per-matrix and aggregate metrics
<code>dataset_evaluation.py</code>	PCA, KMeans, diversity
<code>hyperparameter_search.py</code>	Grid search over (α, δ)
<code>synthetic_od.npy</code>	Output array shape $(K_{\text{samples}}, 24, 24)$
<code>dataset_metrics.json</code>	Evaluation summary

Example command (1M samples).

```
python generate_od_dataset.py \
  --input EstimatedODMatrix.npy \
  --samples 1000000 --oversample-factor 10 \
  --alpha 500 --perturbation 0 \
  --gamma-shape 0.5 --gamma-scale 1.0 \
  --epsilon-beta 1.0 --epsilon-lambda 500 \
  --memmap-save --skip-plots --skip-base-analysis
```

Chapter 2

Turning-Count Observation and Dataset Construction

2.1 Network representation and turning movements

The road network is parsed from `sioux-falls.net.xml` (SUMO format) into a directed graph $\mathcal{G} = (\mathcal{V}, \mathcal{E})$ using NetworkX. **No traffic simulation** is used—only static topology and connection rules: $|\mathcal{V}| = 24$ junctions, $|\mathcal{E}| = 76$ edges, 384 SUMO `<connection>` elements. Implementation: `src/data/network_parser.py`.

A turning movement is a consecutive edge pair $(e^{\text{in}}, e^{\text{out}})$ at junction v :

$$\omega(e^{\text{in}}) = v = \alpha(e^{\text{out}}), \quad e^{\text{in}} \neq e^{\text{out}}, \quad (2.1)$$

where $\omega(\cdot)$ and $\alpha(\cdot)$ denote head and tail junctions. Admissible movements yield $M = 178$ indices stored in `turning_id_map` and exported as `turning_movements.csv`. Implementation: `src/data/turning_movements.py`.

2.1.1 Binary vs. probabilistic route assignment

Table 2.1 compares the two forward models; ***K*-shortest-path + logit** (`generate_turning_counts.py`) is the production default.

Table 2.1: Comparison of route assignment and turning-count generation methods.

Criterion	Phase 1: binary shortest path	Phase 2: probabilistic (current)
Route set	Single shortest path π_{ij}	Up to K paths $\mathcal{R}_{ij}$
Route choice	Deterministic	Multinomial logit on costs
$\mathbf{A}_{\text{turn}}$ entries	Binary $\{0, 1\}$	Fractional $[0, 1]$
$\text{rank}(\mathbf{A}_{\text{turn}})$	122 (nullity 454)	174 (nullity 402)
Scalability at 10^6	Full in-memory load	Batched memory-mapped I/O

Phase 1 understates route diversity and lowers $\text{rank}(\mathbf{A}_{\text{turn}})$. Phase 2 retains the linear forward model $\mathbf{x} = \mathbf{A}_{\text{turn}}\mathbf{y}$ while encoding route-choice structure via multinomial logit over a finite route catalog.

2.2 Route enumeration and probabilistic assignment

2.2.1 OD pair indexing

All $N^2 = 576$ zone pairs are indexed in row-major order:

$$\ell(i, j) = (i - 1)N + (j - 1), \quad y_{\ell(i, j)} = T_{ij}. \quad (2.2)$$

Implementation: `src/data/od_pairs.py`.

2.2.2 Route catalog: K shortest simple paths

For each (i, j) , $i \neq j$, define $\mathcal{R}_{ij} = \{r_1, \dots, r_{K_{ij}}\}$, $K_{ij} \leq K$, from `networkx.shortest_simple_paths` in non-decreasing cost order. Route cost:

$$c_r = \sum_{e \in r} w_e, \quad w_e \in \{L_e, T_e\}. \quad (2.3)$$

Default $K = 5$, $\theta = 0.1$. Catalog saved as `route_catalog.pkl`. Implementation: `src/data/k_shortest_paths.py`.

Table 2.2: Route enumeration and route choice: pros and cons.

Method	Pros	Cons
K shortest simple paths (chosen)	Cost-ordered; simulation-free; fixed $\mathbf{A}_{\text{turn}}$	K hyperparameter
Multinomial logit (chosen)	Closed form; one parameter θ	IIA; no congestion
User equilibrium	Realistic splits	Nonlinear; breaks fixed $\mathbf{A}_{\text{turn}}$
Simulation-based	Congestion-aware	Slow at 10^6 labels

2.2.3 Multinomial logit route choice

Given costs $\{c_r\}$ on $\mathcal{R}_{ij}$:

$$U_r = -\theta c_r, \quad (2.4)$$

$$P_r = \frac{\exp(-\theta c_r)}{\sum_{r' \in \mathcal{R}_{ij}} \exp(-\theta c_{r'})}, \quad (2.5)$$

computed via stable softmax. Small θ increases route diversity and $\text{rank}(\mathbf{A}_{\text{turn}})$; large θ collapses toward Phase 1. Implementation: `src/data/route_choice.py`.

2.3 Fractional assignment matrix and forward model

The turn assignment matrix $\mathbf{A}_{\text{turn}}$ maps OD demand $\mathbf{y} \in \mathbb{R}_{\geq 0}^{N^2}$ to expected turning volumes $\mathbf{x} \in \mathbb{R}_{\geq 0}^M$. It is built *once* from static network topology and route-choice probabilities, then reused for every synthetic OD replicate. This section gives the full construction procedure; the forward model that applies $\mathbf{A}_{\text{turn}}$ to $\mathbf{y}$ is stated at the end.

2.3.1 Inputs, outputs, and indexing conventions

Inputs.

- Parsed network $\mathcal{G}$ and turning index $\{1, \dots, M\}$ with $M = 178$ (§2.1).
- OD column index $\ell(i, j)$ from (2.2) for all $i \neq j$ (§2.2).
- Route catalog $\{\mathcal{R}_{ij}\}$ and logit probabilities $\{P_r\}_{r \in \mathcal{R}_{ij}}$ (§2.2).

Output. Matrix $\mathbf{A}_{\text{turn}} \in [0, 1]^{M \times N^2}$, stored as `A_turn.npy` with shape (178, 576) in `float32`. Row m corresponds to turning movement m ; column $\ell(i, j)$ corresponds to OD pair (i, j) . Self-loops ($i = j$) are omitted from routing and receive zero columns.

Matrix semantics. Entry $A_{\text{turn}, m, \ell(i, j)}$ is the *expected fraction* of a unit trip from i to j that traverses turn m , after multinomial logit route choice over $\mathcal{R}_{ij}$. Columns are not constrained to sum to 1 because a route may use several turns; each column sum equals the expected number of distinct turns per trip for that OD pair.

2.3.2 Turn incidence from a route path

A route is an ordered edge list $r = (e_1, e_2, \dots, e_L)$ with $L \geq 2$. At each internal step $k = 1, \dots, L - 1$, consecutive edges (e_k, e_{k+1}) define a candidate turn at junction $\omega(e_k) = \alpha(e_{k+1})$. Let $\Theta_r \subset \{1, \dots, M\}$ be the set of turn indices realized on r :

$$\Theta_r = \{m : m = \text{id}(e_k, e_{k+1}), k = 1, \dots, L - 1\}, \quad (2.6)$$

where $\text{id}(\cdot, \cdot)$ looks up $(e^{\text{in}}, e^{\text{out}})$ in `turning_id_map`. Pairs not in the map (e.g. degenerate paths) are skipped. The **binary turn-incidence vector** is

$$b_{r, m} = \mathbb{1}[m \in \Theta_r], \quad \mathbf{b}_r \in \{0, 1\}^M. \quad (2.7)$$

Implementation: `route_turn_indices` and `route_turn_vector` in `src/data/fractional_assignment.py`. The Phase 1 binary matrix uses the same incidence rule on a single shortest path per OD (`src/data/build_assignment_matrix.py`).

2.3.3 Column assembly for each OD pair

For each origin–destination pair (i, j) with $i \neq j$, let $\ell = \ell(i, j)$. Initialize column $\mathbf{a}_\ell = \mathbf{0} \in \mathbb{R}^M$. For every route $r \in \mathcal{R}_{ij}$ with probability P_r from (2.5),

$$a_{\ell, m} \leftarrow a_{\ell, m} + P_r b_{r, m}, \quad m = 1, \dots, M. \quad (2.8)$$

Equivalently, in vector form $\mathbf{a}_\ell = \sum_{r \in \mathcal{R}_{ij}} P_r \mathbf{b}_r$. Stacking columns yields the fractional assignment matrix:

$$A_{\text{turn}, m, \ell} = \sum_{r \in \mathcal{R}_{ij}} P_r b_{r, m}. \quad (2.9)$$

If $\mathcal{R}_{ij} = \emptyset$ (unreachable pair), column ℓ remains zero. With $K = 5$ and $\theta = 0.1$ on Sioux Falls, $\mathbf{A}_{\text{turn}}$ has 7,326 nonzeros (vs. 1,324 for the binary shortest-path matrix $\mathbf{A}_{\text{turn}}^{\text{bin}}$).

2.3.4 Binary baseline for comparison

Phase 1 assigns one shortest path π_{ij} per OD pair (`src/data/route_assignment.py`) and sets

$$A_{\text{turn}, m, \ell(i, j)}^{\text{bin}} = \mathbb{1}[m \in \Theta_{\pi_{ij}}], \quad (2.10)$$

so entries are in $\{0, 1\}$. The production pipeline always builds both $\mathbf{A}_{\text{turn}}^{\text{bin}}$ and $\mathbf{A}_{\text{turn}}$, compares ranks via `src/data/assignment_rank.py`, and persists only the probabilistic $\mathbf{A}_{\text{turn}}$ for turning-count generation.

2.3.5 Validation checks

After construction, `validate_fractional_matrix` verifies:

- $0 \leq A_{\text{turn},m,\ell} \leq 1$ for all (m, ℓ) ;
- column sums $s_\ell = \sum_m A_{\text{turn},m,\ell}$ satisfy $s_\ell > 0$ for routed OD pairs (typical mean $\bar{s} \approx 3.9$ turns per trip on Sioux Falls);
- probabilities within each $\mathcal{R}_{ij}$ sum to 1 (checked at route-choice stage).

Rank and condition number are reported in Table 2.3; $\text{rank}(\mathbf{A}_{\text{turn}}) = 174$ vs. 122 for $\mathbf{A}_{\text{turn}}^{\text{bin}}$.

Algorithm 2: Construction of the turn assignment matrix $\mathbf{A}_{\text{turn}}$

1. **Parse network:** load `sioux-falls.net.xml`; build $\mathcal{G}$ and enumerate $M = 178$ admissible turns from SUMO `<connection>` rules; export `turning_movements.csv`
2. **Index OD pairs:** map each (i, j) , $i \neq j$, to column $\ell(i, j)$; $N^2 = 576$ columns
3. **Enumerate routes:** for each (i, j) , compute up to K shortest simple paths $\mathcal{R}_{ij}$ in non-decreasing cost order; save `route_catalog.pkl`
4. **Route choice:** for each $\mathcal{R}_{ij}$, compute costs c_r , utilities $U_r = -\theta c_r$, and probabilities P_r via stable softmax (2.5)
5. **Turn incidence:** for each route $r = (e_1, \dots, e_L)$, form Θ_r from consecutive edge pairs (2.6); set $b_{r,m} = \mathbb{1}[m \in \Theta_r]$
6. **Assemble $\mathbf{A}_{\text{turn}}$:** for each $\ell = \ell(i, j)$, set $A_{\text{turn},:, \ell} = \sum_{r \in \mathcal{R}_{ij}} P_r \mathbf{b}_r$ (2.9)
7. **Validate and compare:** check entries in $[0, 1]$; build $\mathbf{A}_{\text{turn}}^{\text{bin}}$ from shortest paths; report $\text{rank}(\mathbf{A}_{\text{turn}})$ and $\text{rank}(\mathbf{A}_{\text{turn}}^{\text{bin}})$
8. **Persist:** save `A_turn.npy` (shape 178×576 , `float32`)

Figure 2.1: Turn assignment matrix generation (`generate_turning_counts.py`, stages 1–7; core logic in `fractional_assignment.py`).

Implementation: `src/data/fractional_assignment.py` (`build_fractional_assignment_matrix`), orchestrated by `generate_turning_counts.py`.

2.3.6 Forward model

Given a fixed $\mathbf{A}_{\text{turn}}$, the forward model for one replicate and for batches is:

$$\mathbf{x}^{\text{clean}} = \mathbf{A}_{\text{turn}} \mathbf{y}, \quad \mathbf{X}^{\text{clean}} = \mathbf{Y} \mathbf{A}_{\text{turn}}^\top. \quad (2.11)$$

2.3.7 Identifiability and rank analysis

Table 2.3: Assignment matrix rank comparison ($K = 5$, $\theta = 0.1$).

Quantity	$\mathbf{A}_{\text{turn}}^{\text{bin}}$	$\mathbf{A}_{\text{turn}}$ (Phase 2)
$\text{rank}(\mathbf{A}_{\text{turn}})$	122	174
Nullity	454	402
$\kappa(\mathbf{A}_{\text{turn}})$	≈ 14.8	$\approx 1.2 \times 10^{12}$

Rank increases by 52, improving information content for Chapter 3. The inverse problem remains underdetermined ($\mathcal{N} = 402$). Implementation: `src/data/assignment_rank.py`.

2.4 Observation noise, dataset assembly, and scalability

2.4.1 Observation noise models

Applied after clean synthesis. Real loop detectors and camera counts are noisy.

No noise. $\mathbf{x}^{\text{obs}} = \mathbf{x}^{\text{clean}}$.

Gaussian noise.

$$x_m^{\text{obs}} = x_m^{\text{clean}} + \epsilon_m, \quad \epsilon_m \sim \mathcal{N}(0, \sigma^2 (x_m^{\text{clean}})_+), \quad (2.12)$$

with clipping at zero.

Poisson noise (default).

$$x_m^{\text{obs}} \sim \text{Poisson}(x_m^{\text{clean}}), \quad (2.13)$$

with $\text{Var}(x_m^{\text{obs}} \mid \mathbf{x}^{\text{clean}}) = x_m^{\text{clean}}$. Implementation: `src/data/observation_noise.py`.

Table 2.4: Observation noise models: pros and cons.

Model	Pros	Cons
None	Exact labels for sanity checks	Optimistic ML performance
Gaussian	Simple; additive	Negative counts without clipping
Poisson (chosen)	Standard for count data	Not for overdispersed sensors
Negative binomial	Handles overdispersion	Extra parameter

2.4.2 End-to-end pipeline and batched generation

Network $\rightarrow K$ **paths** $\rightarrow$ **Logit** $\rightarrow \mathbf{A}_{\text{turn}}$ (Alg. 2.1) $\rightarrow \mathbf{X}^{\text{clean}} = \mathbf{Y}\mathbf{A}_{\text{turn}}^\top \rightarrow$ **Poisson noise** $\rightarrow \mathbf{X}^{\text{obs}}$

Figure 2.2: Chapter 2 turning-count pipeline (`generate_turning_counts.py`).

Loading all 10^6 matrices into RAM caused OOM on a 15 GB machine. The fix (`src/data/batch_turning.py`) uses memory-mapped I/O, batched $\mathbf{X} = \mathbf{Y}\mathbf{A}_{\text{turn}}^\top$, and streaming statistics. Benchmark: 10^6 samples in ≈ 16 s at $B = 20,000$, peak RAM ≈ 60 MB per batch.

Algorithm 3: Batched turning-count generation ($K_{\text{samples}} = 10^6$)

1. Run Algorithm 2.1 once to obtain $\mathbf{A}_{\text{turn}}$
2. Memory-map `synthetic_od.npy`; never load full $\mathbf{Y}$
3. For each batch of size B : $\mathbf{Y}^{(b)} \rightarrow \mathbf{X}^{\text{clean},(b)} = \mathbf{Y}^{(b)}\mathbf{A}_{\text{turn}}^\top$; apply Poisson; write rows
4. Streaming mean/std; sampled pairwise diversity

Figure 2.3: Memory-safe batching (`src/data/batch_turning.py`).

2.4.3 Validation, sensitivity, and evaluation highlights

Forward consistency: $\|\mathbf{x}^{\text{clean}} - \mathbf{A}_{\text{turn}}\mathbf{y}\|_\infty < 10^{-3}$ per batch. Sensitivity grid over (K, θ) via `--run-sensitivity`. On 10^6 synthetics: rank = 174; $\bar{d}_{\text{OD}} \approx 545$; $\bar{d}_{\text{turn}} \approx 894$; mean inter-turn correlation ≈ 0.01 .

2.5 Implementation reference

```
python generate_turning_counts.py \
--network sioux-falls.net.xml \
```

Table 2.5: Chapter 2: key modules (`src/data/`) and deliverables.

Module / file	Role
<code>generate_turning_counts.py</code>	CLI orchestrator
<code>k_shortest_paths.py</code>	K shortest paths per OD
<code>route_choice.py</code>	Multinomial logit
<code>build_assignment_matrix.py</code>	Binary baseline $\mathbf{A}_{\text{turn}}^{\text{bin}}$
<code>fractional_assignment.py</code>	Fractional $\mathbf{A}_{\text{turn}}$ (Algorithm 2.1)
<code>batch_turning.py</code>	Memory-mapped batched generation
<code>observation_noise.py</code>	None / Gaussian / Poisson
<code>A_turn.npy</code>	(178, 576) fractional matrix
<code>turning_counts.npy</code>	(K_{samples} , 178) clean counts
<code>turning_counts_noisy.npy</code>	Poisson observations
<code>route_catalog.pkl</code>	Routes + logit probabilities

```
--od outputs/od_generator/synthetic_od_1m.npy \
--output-dir outputs/turning_counts_1m \
--k_paths 5 --theta 0.1 --noise poisson \
--batch-size 20000 --skip-plots
```

Chapter 3

OD Estimation: Information Analysis, Models, and Benchmarking

3.1 Inverse problem and design principle

Given turning-count observations $\mathbf{X} \in \mathbb{R}^{K_{\text{samples}} \times M}$, $M = 178$, learn

$$f : \mathbb{R}^M \rightarrow \mathbb{R}^{576}, \quad \hat{\mathbf{y}}^{(k)} = f(\mathbf{x}^{\text{obs},(k)}), \quad (3.1)$$

approximating flattened OD $\mathbf{y}^{(k)}$. The forward model from Chapter 2 is

$$\mathbf{x}^{\text{clean}} = \mathbf{A}_{\text{turn}} \mathbf{y}, \quad \mathbf{x}^{\text{obs}} \sim \text{Noise}(\mathbf{x}^{\text{clean}}), \quad (3.2)$$

with $\text{rank}(\mathbf{A}_{\text{turn}}) = 174$, nullity $\mathcal{N} = 402$. The per-sample inverse map is underdetermined.

$(\mathbf{X}, \mathbf{Y}) \rightarrow \text{statistics} / \text{PCA} / \text{MI} / \text{rank}(\mathbf{A}_{\text{turn}}) \rightarrow \text{Ridge, PLS} \rightarrow \text{RF, XGBoost} \rightarrow \text{MLP, autoencoder} \rightarrow \text{SHAP} + \text{leaderboard}$

Figure 3.1: Chapter 3 pipeline (`train_od_estimator.py`). Classical models establish an information ceiling before deep models.

Design principle. Do **not** start with deep learning. First quantify recoverable OD information via $\text{rank}(\mathbf{A}_{\text{turn}})$, PCA dimensionality, and mutual information $\text{MI}(x_m, y_\ell)$. If Ridge/PLS achieve low error, deep models test noise robustness—not representational capacity.

Underdetermined system vs. training set size. These are distinct: (i) each sample has 178 observations for 576 unknowns ($\mathcal{N} = 402$); (ii) K_{samples} labeled pairs help *learn* f , but do not remove per-sample ambiguity without regularization or priors.

3.2 Dataset and information content analysis

Inputs $\mathbf{X}_{k,m} = x_m^{(k)}$ from `turning_counts.npy`; targets $\mathbf{Y}_{k,\ell} = y_\ell^{(k)}$ from flattened `synthetic_od*.npy`. Implementation: `src/ml/dataset.py`.

Per-feature statistics (mean, std, CV), correlation matrices, and PCA ($n_{90\%}$ components for 90% variance). Mutual information for sampled turn–OD pairs:

$$\text{MI}(x_m, y_\ell) = \iint p(x_m, y_\ell) \log \frac{p(x_m, y_\ell)}{p(x_m)p(y_\ell)} dx_m dy_\ell. \quad (3.3)$$

Outputs: `important_turns.csv`, `feature_importance.csv`, `figures/pca_X.png`, `singular_value_spectrum`. Implementation: `src/ml/information_analysis.py`.

3.3 Model zoo and selection rationale

Table 3.1: Model tiers: evaluation order, pros, and cons.

Tier	Models	Pros	Cons
Classical	Linear, Ridge, PLS	Fast; interpretable; PLS for $M < N^2$	Linear; Lasso slow at 576 outputs
Tree	RF, ExtraTrees, XGBoost	Nonlinear; robust	Slow; 576-output regression
Neural	MLP, Residual MLP, AE	Flexible; latent $z \in \mathbb{R}^{16}$	PyTorch; overfitting risk

Why PLS. Partial Least Squares finds latent directions maximizing $\text{cov}^2(\mathbf{X}\mathbf{w}_h, \mathbf{Y}\mathbf{q}_h)$ —well suited to collinear turning counts and multi-target regression.

Why Ridge before MLP. On clean Phase 2 data, $\mathbf{x} = \mathbf{A}_{\text{turn}}\mathbf{y}$ is exactly linear. Ridge with cross-validated α estimates the best regularized linear inverse for $\mathcal{N} = 402$.

Autoencoder architecture. Optional two-stage: (i) OD autoencoder $\mathbf{y} \rightarrow \mathbf{z} \in \mathbb{R}^{16} \rightarrow \hat{\mathbf{y}}$; (ii) map $\mathbf{x} \rightarrow \mathbf{z}$, decode. Reduces output dimensionality when nullity is large.

3.4 Physics constraints, loss functions, and evaluation metrics

Non-negativity: $\hat{\mathbf{y}} \leftarrow \max(\hat{\mathbf{y}}, \mathbf{0})$. Neural training may add production/attraction penalties:

$$\mathcal{L}_{\text{phys}} = \text{MSE}(\hat{\mathbf{y}}, \mathbf{y}) + \lambda_p \|\hat{\mathbf{p}} - \mathbf{p}\|_2^2 + \lambda_a \|\hat{\mathbf{a}} - \mathbf{a}\|_2^2. \quad (3.4)$$

3.4.1 Loss functions

Table 3.2: Loss functions for OD recovery.

Loss	Formula (single sample)	Use case
MSE	$\ \hat{\mathbf{y}} - \mathbf{y}\ _2^2$	Default supervised
MAE	$\ \hat{\mathbf{y}} - \mathbf{y}\ _1$	Robust to outliers
Forward-consistency	$\ \mathbf{x} - \mathbf{A}_{\text{turn}}\hat{\mathbf{y}}\ _2^2$	Physics-informed
Combined	$\text{MSE} + \lambda \ \mathbf{x} - \mathbf{A}_{\text{turn}}\hat{\mathbf{y}}\ _2^2$	Deployment

Evaluation metrics on held-out test data:

$$\text{MAE} = \frac{1}{N^2} \sum_{\ell} |\hat{y}_{\ell} - y_{\ell}|, \quad \text{RMSE} = \sqrt{\frac{1}{N^2} \sum_{\ell} (\hat{y}_{\ell} - y_{\ell})^2}, \quad (3.5)$$

$$R^2 = 1 - \frac{\sum_{\ell} (\hat{y}_{\ell} - y_{\ell})^2}{\sum_{\ell} (y_{\ell} - \bar{y})^2}, \quad \rho_{\text{OD}} = \text{corr}(\mathbf{y}, \hat{\mathbf{y}}), \quad (3.6)$$

$$\|\mathbf{T} - \hat{\mathbf{T}}\|_F = \sqrt{\sum_{i,j} (T_{ij} - \hat{T}_{ij})^2}. \quad (3.7)$$

Production and attraction errors compare row and column sums. Implementation: `src/ml/metrics.py`.

3.5 Training protocol, robustness, and explainability

80/10/10 train/validation/test split (seed-controlled). Standardize $\mathbf{X}$ on training statistics. Default `max_samples=50000` for tractable benchmarking; Ridge/PLS can use larger K_{samples} (e.g. 100,000–1,000,000).

Robustness. Train on clean $\mathbf{X}^{\text{clean}}$; evaluate on Poisson-noisy $\mathbf{X}^{\text{obs}}$ at 0%, 2%, 5%, 10% levels.

SHAP. Tree-model SHAP values identify influential turning movements (`figures/shap_summary.png`). Implementation: `src/ml/explainability.py`.

Table 3.3: Practical limits for $K_{\text{samples}} = 10^6$ on 15 GB RAM.

Model	1M feasible?	Notes
Ridge, Linear, PLS	Yes	~3 GB float32 arrays; minutes
Random Forest, XGBoost	Slow	Hours; high memory
Lasso, Elastic Net	No (default)	576 separate sparse fits
MLP, Autoencoder	Slow	Use GPU; batch training

Training at scale.

3.6 Benchmark leaderboard and deliverables

Table 3.4: Chapter 3 deliverables (`outputs/ml/`).

File	Content
<code>best_model.pkl</code>	Best model by test MAE
<code>benchmark_results.csv</code>	Leaderboard (MAE, RMSE, R^2 , marginals)
<code>feature_importance.csv</code>	Ranked turning movements
<code>predictions.npy</code>	Test-set $\hat{\mathbf{Y}}$
<code>evaluation_report.md</code>	Summary report

Table 3.5: Chapter 3 modules (`src/ml/`).

Module	Role
<code>train_od_estimator.py</code>	CLI orchestrator
<code>dataset.py</code>	Load, split, standardize
<code>information_analysis.py</code>	Stats, PCA, MI, rank
<code>models_classical.py</code>	Linear, Ridge, PLS, trees
<code>models_neural.py</code>	MLP, residual MLP, autoencoder
<code>benchmark.py</code>	Leaderboard and report

```
# Full benchmark (use ODEstimation env Python)
/home/chetan/anaconda3/envs/ODEstimation/bin/python train_od_estimator.py \
  --turning outputs/turning_counts_1m/turning_counts.npy \
  --od outputs/od_generator/synthetic_od_1m.npy \
  --max-samples 50000
```

```
# Autoencoder on 100k samples
/home/chetan/anaconda3/envs/ODEstimation/bin/python train_od_estimator.py \
  --model autoencoder --max-samples 100000 --skip-analysis \
  --output-dir outputs/ml_autoencoder_100k
```

3.7 Future work and limitations

Table 3.6: Planned extensions beyond this document.

Topic	Description
Real-data validation	Held-out counts / partial OD
Overdispersed noise	Negative binomial on $\mathbf{x}^{\text{obs}}$
User equilibrium	Nonlinear forward model
Uncertainty quantification	Bayesian / ensemble CIs on $\mathbf{y}$
Deployment	Field turning-count inference pipeline

Current limitations.

- Single observed $\mathbf{T}^{\text{base}}$; no temporal dynamics;
- Logit route choice without congestion feedback;
- Nullity $\mathcal{N} = 402$ limits unique OD recovery without priors;
- Full 1M benchmark leaderboard not yet populated; subsampling used by default;
- Use ODEstimation conda env (not base python) for PyTorch/matplotlib compatibility.

Conclusion

Chapter 1 produces diverse, structurally valid synthetic OD matrices via a distance-dependent gravity prior on full off-diagonal support, Dirichlet sampling, Gamma heavy-tailed reweighting, IPF balancing, and dataset quality filters—scalable to 10^6 samples. Chapter 2 constructs turning-count observations through a route-choice-aware fractional operator $\mathbf{A}_{\text{turn}}$ (rank = 174, up from 122 under binary assignment), with optional Poisson noise and memory-mapped batch generation. Chapter 3 implements information analysis, classical baselines (Ridge, PLS), tree models, optional neural estimators, and a benchmark leaderboard via `src/ml/` and `train_od_estimator.py`. The three-chapter structure separates data generation from inverse estimation while maintaining consistent notation from $\mathbf{T}^{\text{base}}$ through $\hat{\mathbf{Y}}$.

Bibliography

- [1] R. Furness, *Trip forecasting*, Traffic Engineering and Control, 1962.
- [2] W. E. Deming and F. F. Stephan, *On a least squares adjustment of a sampled frequency table when the expected marginal totals are known*, Annals of Mathematical Statistics, 1940.
- [3] J. de D. Ortúzar and L. G. Willumsen, *Modelling Transport*, 4th ed., Wiley, 2011.
- [4] E. Cascetta, *Estimation of trip matrices from traffic counts and survey data*, Transportation Research Part B, 1984.